COL1000: Introduction to Programming

Lecture 8

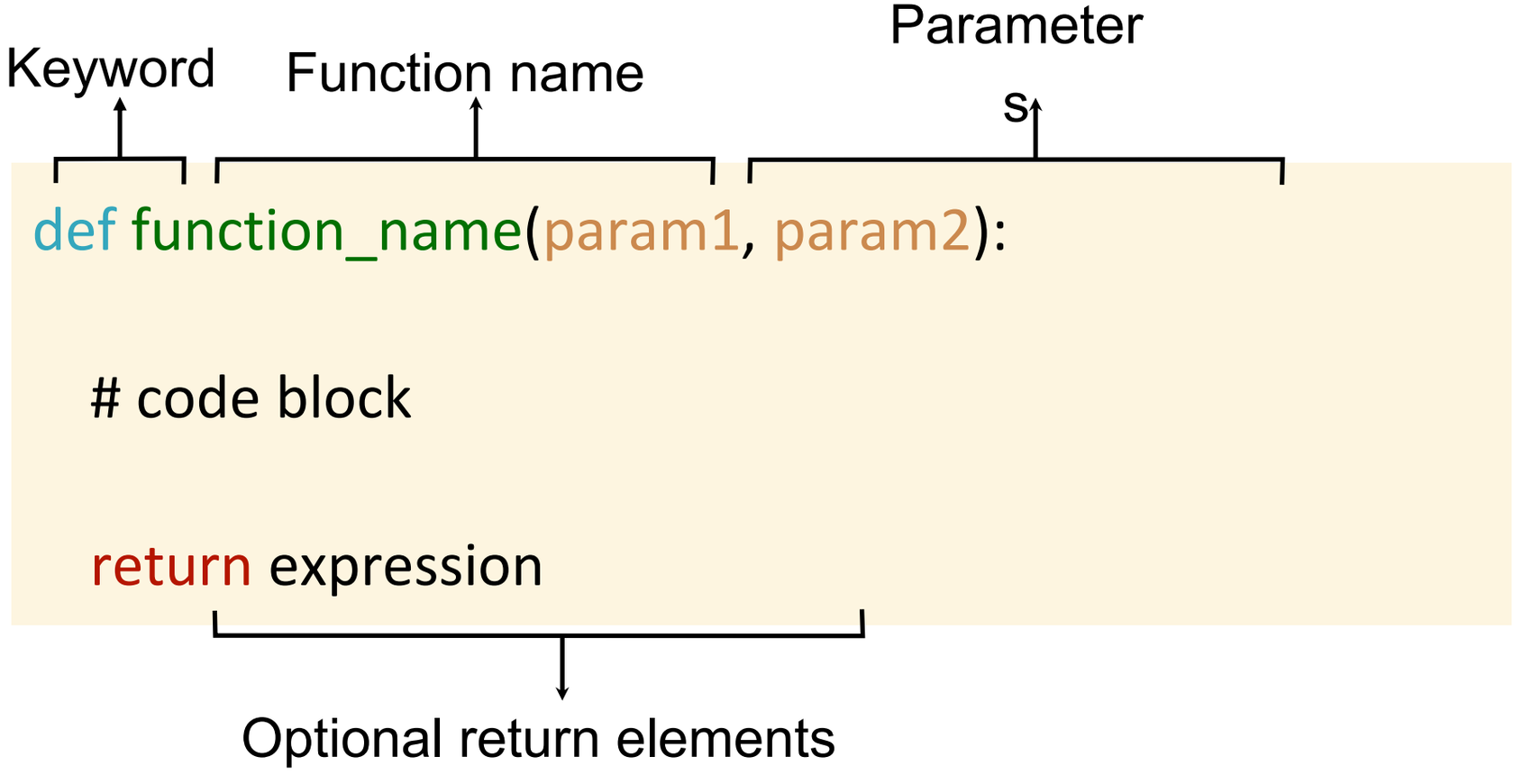
Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi

Functions in Python

A **function** is a **block of reusable code** that performs a specific task.

Why use functions?

- Organize code
- Avoid repetition
- Make code reusable and readable



Functions in Python

A **function** is a **block of reusable code** that performs a specific task.

Why use functions?

- Organize code
- Avoid repetition
- Make code reusable and readable

Defining function

```
def function_name(param1, param2):
    # code block
    return expression
```

Calling a function

```
function_name(input1, input2)
```

Python function to print max

Program

```
def my_max(x, y):
    if (x > y):
        print(x)
    else:
        print(y)

my_max(5, 10)
```

Output

```
10
```

Python function to return max

Program

```
def my_max(x, y):
    if (x > y):
        return x
    else:
        return y

Z = my_max(5, 10)
print(Z)
```

Output

```
10
```

Predict the Output

Program

```
def my_max(x, y):
    if (x > y):
        print(x)
    else:
        print(y)

Z = my_max(5, 10)
print(Z)
```

Output

```
10
None
```

Predict the Output

Program

```
def my_max(x, y):
    if (x > y):
        print("Max: ",x)
        return x
    else:
        print("Max: ",y)
        return y

Z = my_max(5, 10)
print(Z)
```

Output

```
Max:  10
10
```

Control flow with functions

Program

```
print("Here 1")

def my_max(x, y):
    if (x > y):
        return x
    else:
        return y
    print("Here 2")

print("Here 3")

print(my_max(5, 10))
```

Output

```
Here 1
Here 3
10
```

Functions calling functions

Program

```
def max2(a, b):
    print("Here",a,b)
    if (a>b):
        return a
    else:
        return b

def max3(a, b, c):
    z = max2(a, b)
    return max2(z,c)

print(max3(2,8,7))
```

Output

```
Here 2 8
Here 8 7
8
```

We have already seen functions in Python!

Some inbuilt functions in Python:

print(…) : prints the input, returns None

int(…) : returns an integer

float(…) : returns a float

str(…) : returns a string

bool(…) : returns a bool

len(…) : takes as input a string, returns an integer

Difference between print(x) and return x

print(x) sends text to the screen,

return x sends the value x to the place where the function was called

Counting co-prime pairs using Functions

Program

```
def isCoPrime(a, b):

   for i in range(2, min(a,b)+1):
      if ((a%i==0) and (b%i==0)):
         return _____          False

   return _____                True
```

Do we need to check for all even values of i?

```
n = 5
count = 0

for i in range(1, n+1):
   for j in range(i+1, n+1):
      if isCoPrime(i,j) == True:
         count = count+1

print(count)
```

Counting co-prime pairs using Functions

Program

```
def isCoPrime(a, b):
    if ((a%2==0) and (b%2==0)):
        return _____
    for i in range(3, min(a,b)+1, 2):
        if ((a%i==0) and (b%i==0)):
            return _____
    return _____
```

False

False

True

More efficient algorithm will be covered later!

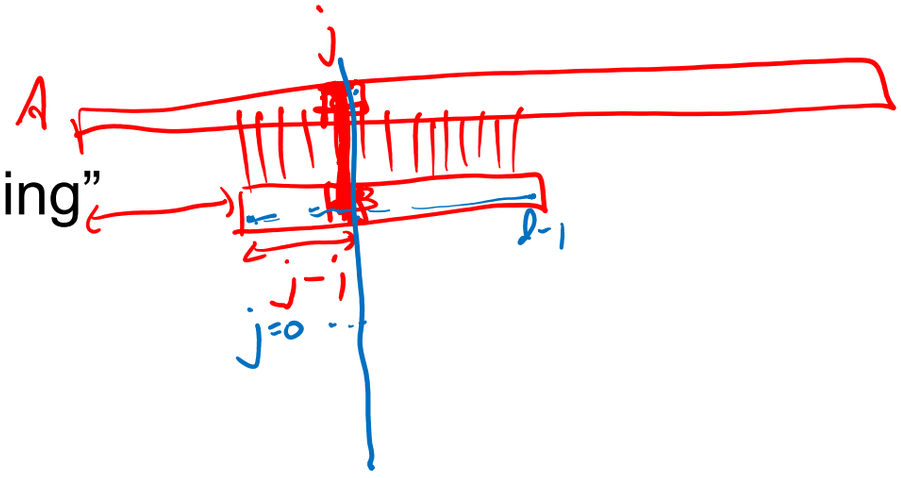
```
n = 5
count = 0

for i in range(1, n+1):
    for j in range(i+1, n+1):
        if isCoPrime(i,j) == True:
            count = count+1

print(count)
```

Exercise

Write a function "check_substring(*A*, *B*)" that takes two strings *A*, *B* as input, and checks if

B is a contiguous substring of *A*.

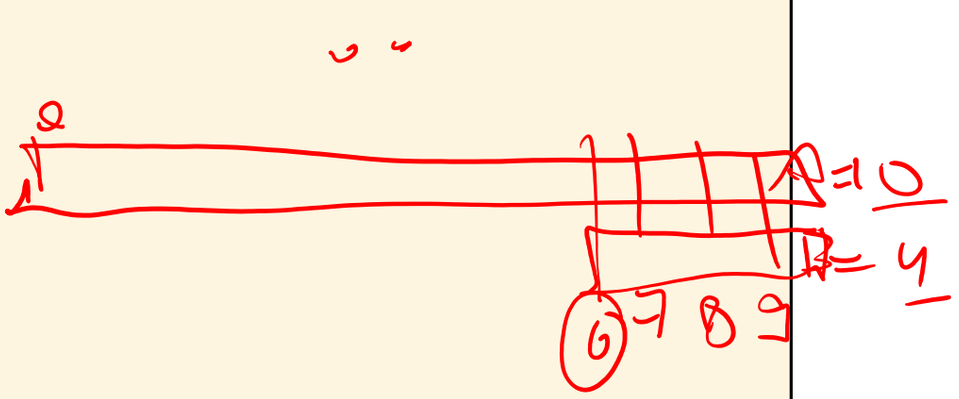
Eg: "gram" is a contiguous substring of "Programming"



Must not use any string libraries other than the ones discussed in class. In particular, don't use ***if (str1 in str2)*** *for this problem.*

Exercise Solution

```
def check_substring(A, B):

    if (len(B) > len(A)):
        return False

    for i in range(_____len(A)-len(B)+1_____):
        if (A[_____i:i+len(B)_____] == B):
            return True

    return False

s1 = "Programming"
s2 = "gram"
print(check_substring(s1, s2))
```

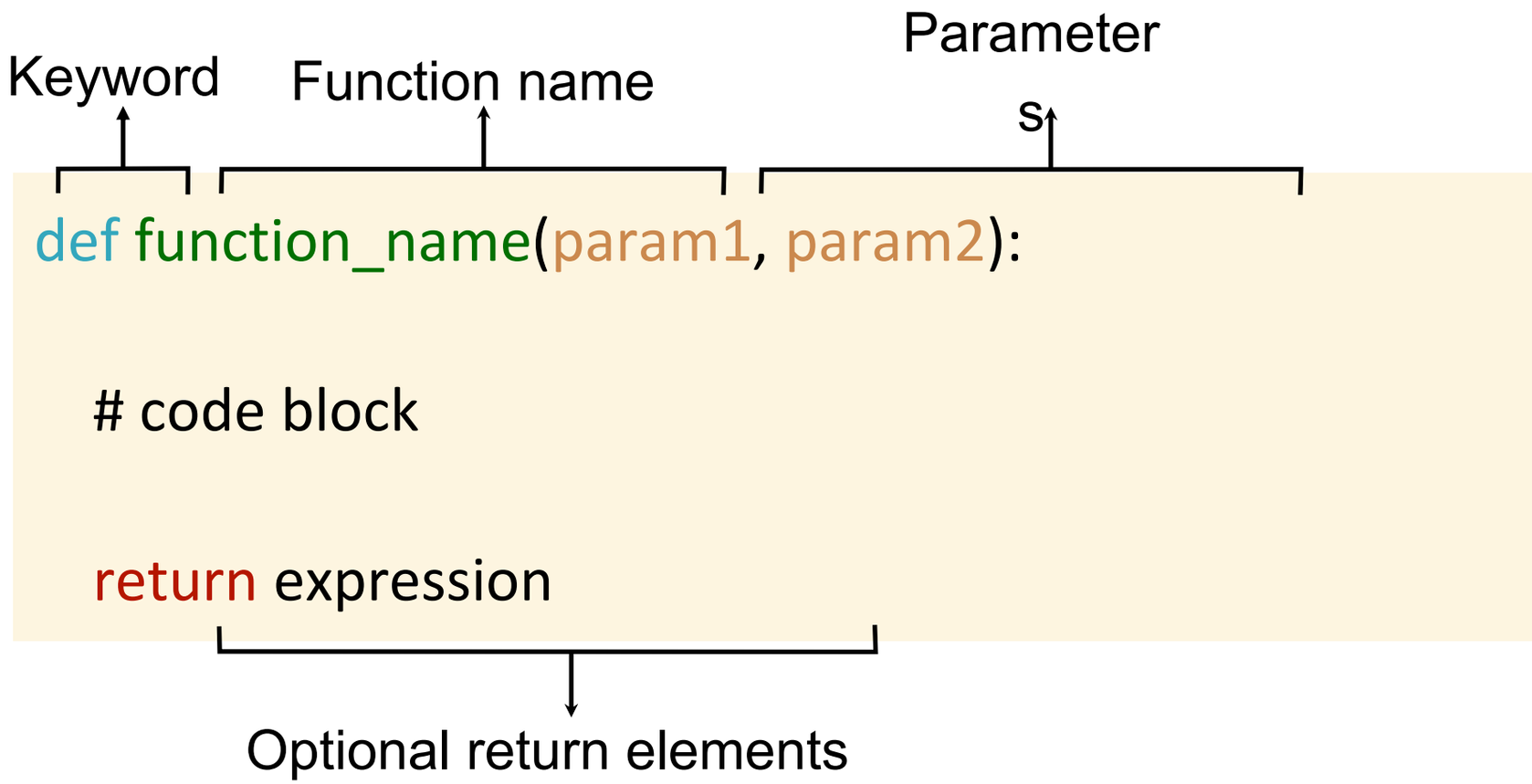


COL1000: Introduction to Programming

Lecture 9

Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi

Functions in Python



Today's lecture

More on Functions:

1. **Recursions**
2. **Variable Scope**

Recursion

Recursion in a function is a programming technique where a function calls itself from within its own code to solve a problem.

This approach breaks a complex problem down into smaller, identical subproblem(s) until it reaches the base case condition.

```
def fib(n):
    if n == 0: return(0)
    elif n == 1: return(1)
    else: return(fib(n-1)+fib(n-2))
```

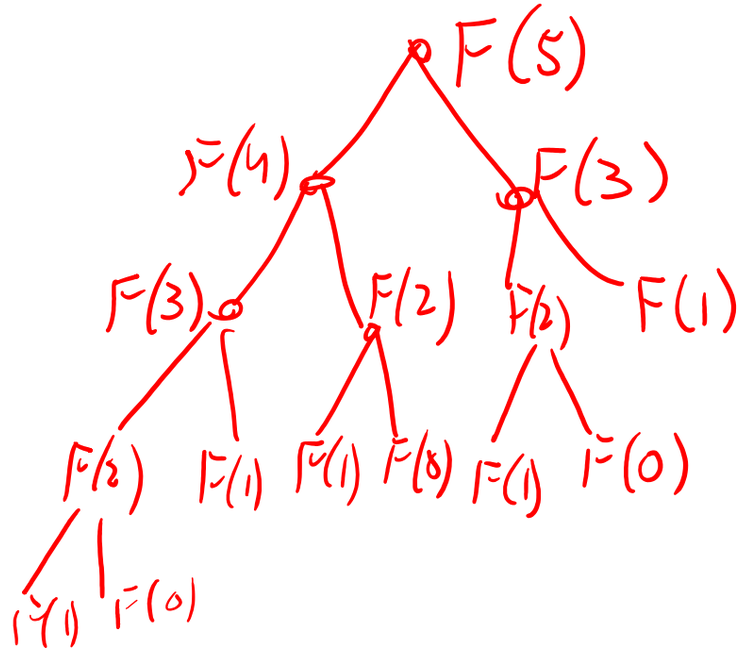
Factorial(n) = n * factorial(n-1) | inductive def

factorial(1) = 1 ←

Fib(n) = Fib(n-1) + Fib(n-2) | n ≥ 2

F(1) = 1

F(0) = 0

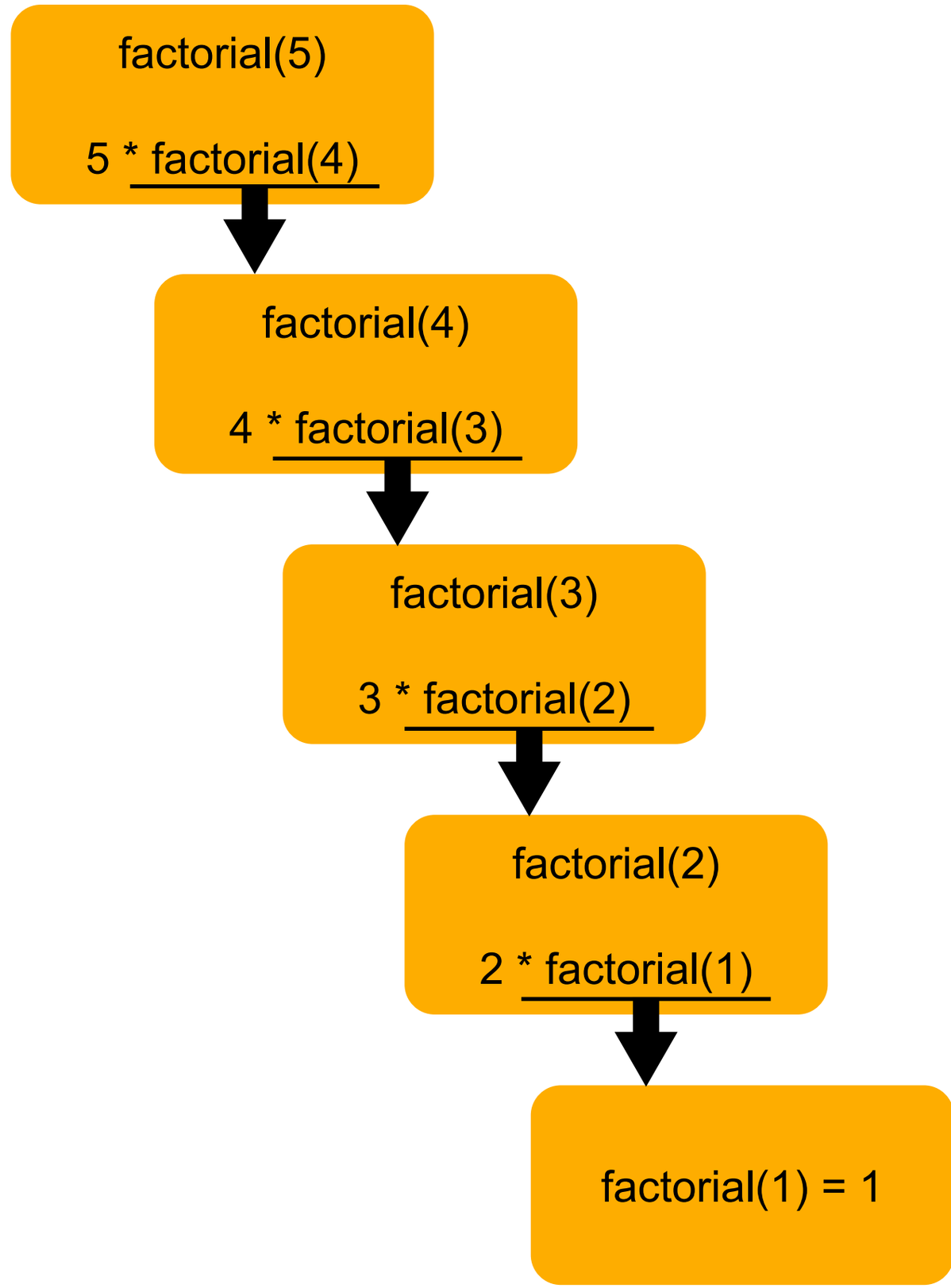


Factorial via Recursion

Program

```
def factorial(n):

    if (n==1):
        return 1
    else:
        return (n * factorial(n-1))


print(factorial(5))
```

In **each** recursive call, Python creates a **new copy of the variables**.

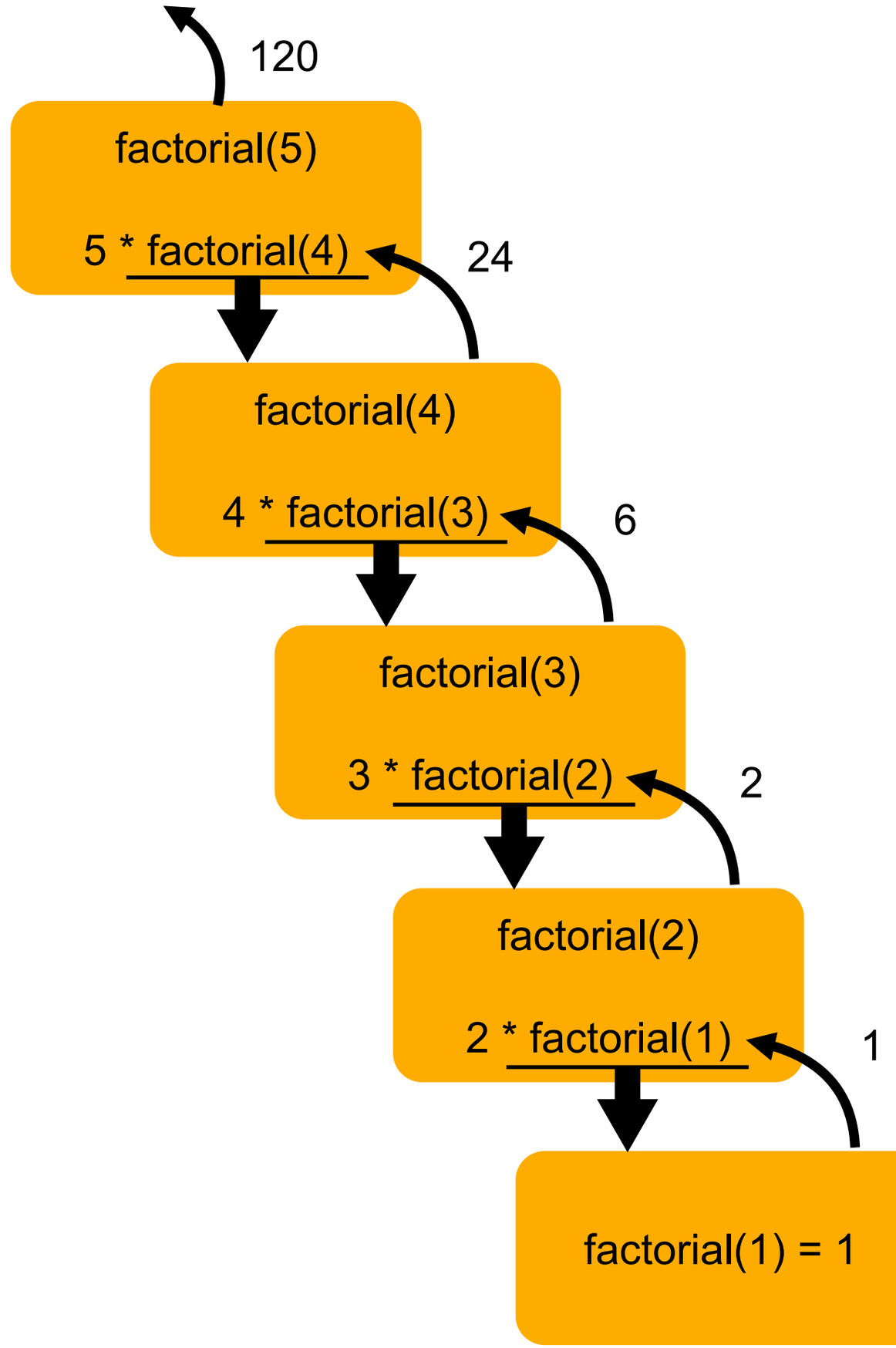


Factorial via Recursion

Program

```
def factorial(n):

    if (n==1):
        return 1
    else:
        return (n * factorial(n-1))


print(factorial(5))
```

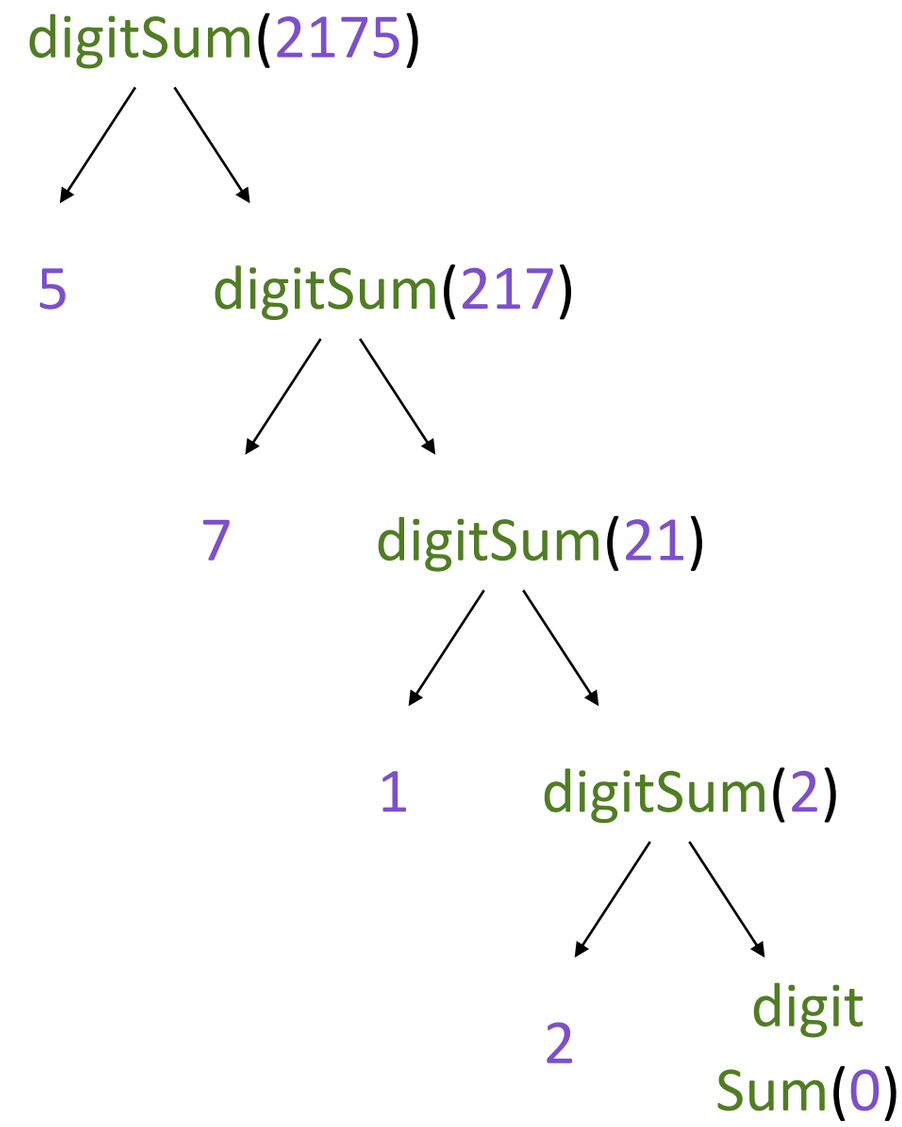
In **each** recursive call, Python creates a **new copy of the variables**.



Computing Sum of Digits

Program

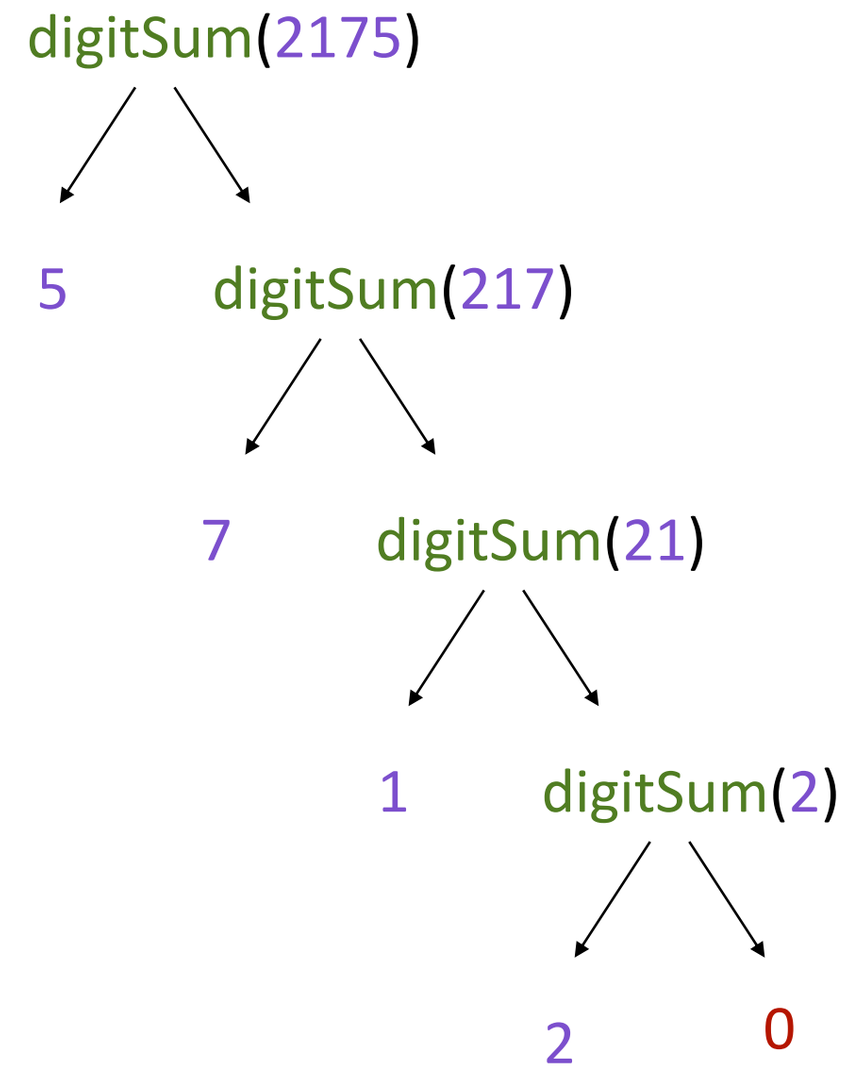
```
def digitSum(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(____last + digitSum(rem)_)


Z = digitSum(2175)
print(Z)
```



Computing Sum of Digits

Program

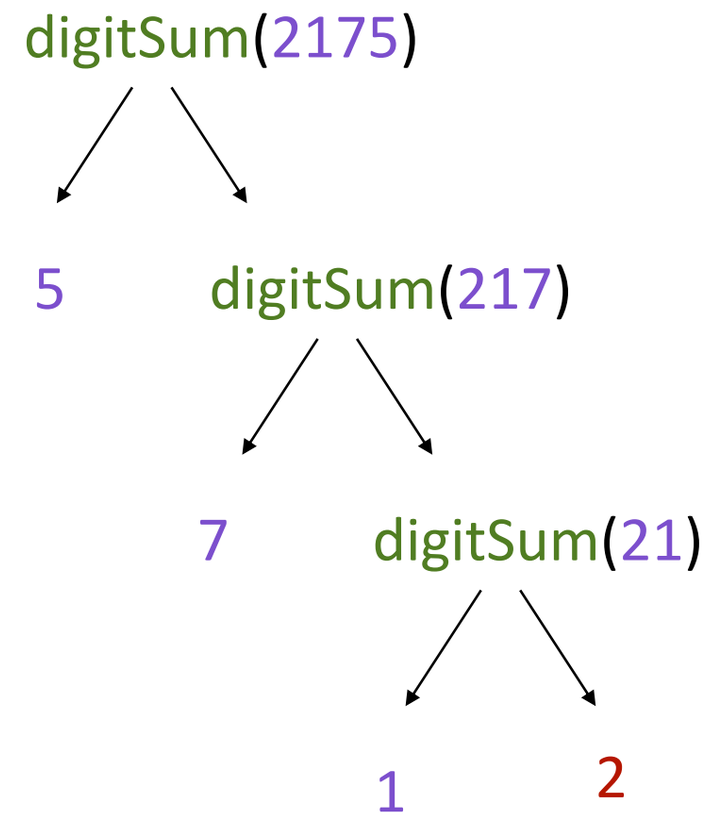
```
def digitSum(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(____last + digitSum(rem)_)


Z = digitSum(2175)
print(Z)
```



Computing Sum of Digits

Program

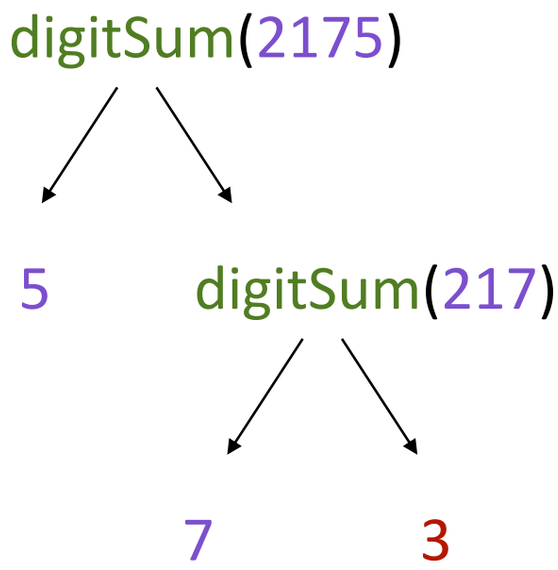
```
def digitSum(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(____last + digitSum(rem)_)


Z = digitSum(2175)
print(Z)
```



Computing Sum of Digits

Program

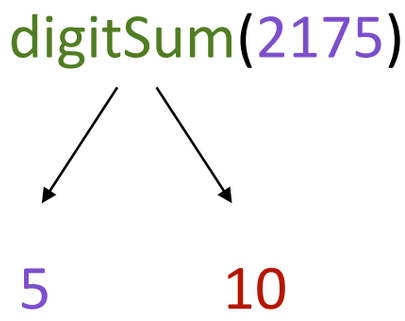
```
def digitSum(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(____last + digitSum(rem)__)

Z = digitSum(2175)
print(Z)
```



Computing Sum of Digits

Program

```
def digitSum(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(____last + digitSum(rem)_)


Z = digitSum(2175)
print(Z)
```



Computing Sum of Digits

Program

```
def digitSum(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(____last + digitSum(rem)_)


Z = digitSum(2175)
print(Z)
```

Z = 15

Variable Scope

Python maintains a **symbol table** storing all variable names defined, and their current values

When a function is called, a fresh symbol table is created for this function call.
- inherits values from the ‘parent’
- the actual parameters are assigned to the formal parameters
- when return statement is encountered, this symbol table is deleted

Variable Scoping: Example 1

Program

```
def f(x):
    sq = x*x

    print("In function")
    print("x=", x, "sq=", sq)
    return sq

x = 3
z = f(x)

print("Outside function")
print("x=", x, "sq=", sq, "z=", z)
```

Output

```
In function
x= 3 sq= 9
Outside function
ERROR!


NameError: name 'sq' is not
defined
```

When Python exits the function, it “deletes” the local variables.

Variable Scoping: Example 2

Program

```
def f(x):
    sq = x*x

    print("In function")
    print("x=", x, "sq=", sq)
    return sq

x = 3
sq = 0
z = f(x)

print("Outside function")
print("x=", x, "sq=", sq, "z=", z)
```

Output

```
In function
x= 3 sq= 9
Outside function
x= 3 sq= 0 z= 9
```

Inside a function,
Python has a **local copy** of variable

More on Variable Scope

- Local scope is the current function's own variables — variables defined inside the function itself.
- Enclosing scope is the scope of the 'ancestor' function when you have nested functions.
- Global scope is outside all functions (i.e. at the top level).

```
x = "global_x"

def outer():
    x = "enclosing_x"

    def inner_1():
        x = "local_x"
        print(x)

    def inner_2():
        print(x)

    inner_1()
    inner_2()

outer()
print(x)
```

```
local_x
enclosing_x
global_x
```

Variable Scoping: Example 3

Program

```
def circle_area(rad):
    ans = pi*(rad**2)
    return ans

def sphere_volume(rad):
    ans = 4*pi*(rad**3)/3
    return ans

pi = 3.142
x = 10

print(circle_area(x))
print(sphere_volume(x))
```

Output

```
314.2
4189.333333333333
```

Here pi is a **global** variable (accessible to both functions).

So, there is no error.

Variable Scoping: Example 4

Program

```
def circle_area(rad):
    ans = pi*(rad**2)
    return ans

def sphere_volume(rad):
    ans = 4*pi*(rad**3)/3
    return ans

x = 10
print(circle_area(x))
pi = 3.142
print(sphere_volume(x))
```

Output

```
Line 2, in circle_area

NameError: name 'pi' is not
defined
```

Here pi is a **global** variable.

But, it is defined after circle_area.

Variable Scoping: Example 5

Program

```
def F(x):
    def G():
        x = "abc"
        print("in G, x =", x)

    print("in F, x =", x)
    x = x*10
    G()
    print("in F, x =", x)

    return x

x = 3
z = F(x)
print("outside, x =", x)
print("outside, z =", z)
```

Output

```
in F, x = 3
in G, x = abc
in F, x = 30
outside, x = 3
outside, z = 30
```

Final remarks on functions (for now …)

To avoid scoping issues, make sure that your functions don't use variables from outside the scope.

```
def addOne(x):
    y = x + 1
    print(y)


x = 3
y = 3

addOne(x)
```

Will print 4

```
def addOne(x):
    y = y + 1
    print(y)


x = 3
y = 3

addOne(x)
```

Will give an error

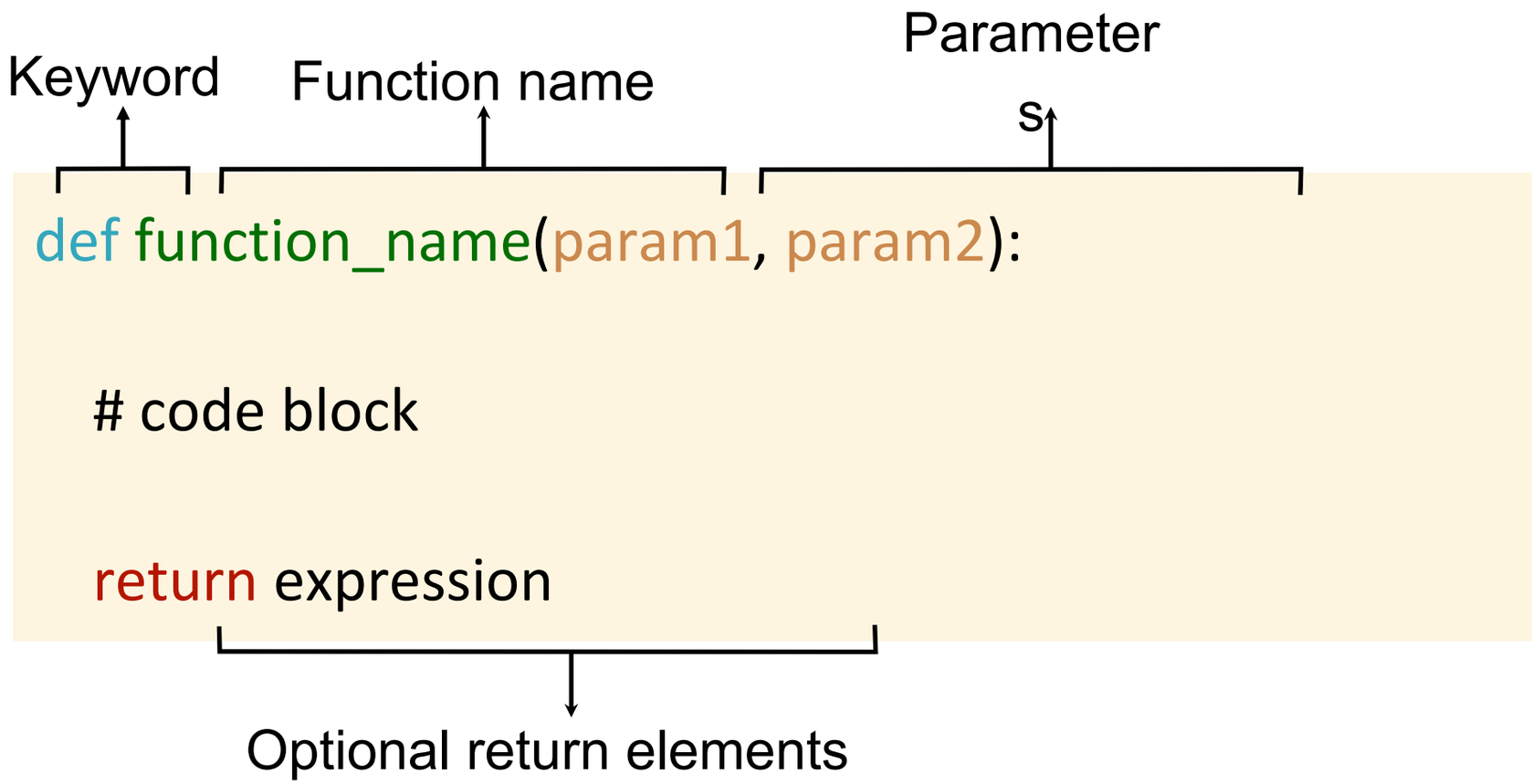
cannot access local variable 'y' where it is not associated with a value

COL1000: Introduction to Programming

Lecture 10

Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi

Functions in Python



Functions Review

```
def is_prime(n):

    if (n == 1):

        return False

    else:

        for i in range(2, n):

            if (n%i == 0):

                return False

        return True
```

```
def is_prime(n):

    if (n == 1):

        return False



    for i in range(2, n):

        if (n%i == 0):

            return False

    return True
```

What happens if
we remove else?

Equivalent

Functions Review: Product of digits

Program

```
def digitProd(x):
    if (x == 0):
        return 0
    else:
        last = x % 10
        rem = x // 10
        return(___last * digitProd(rem)__)


Z = digitProd(2175)
print(Z)
```

Functions Review: Variable scope

```
def addOne(x):
    y = x + 1
    print(y)


x = 3
y = 3

addOne(x)
```

Will print 4

```
def addOne(x):
    y = y + 1
    print(y)


x = 3
y = 3

addOne(x)
```

Will give an error

Here, 'y' is considered a local variable. Error comes as it is not associated with a value

Functions Review: Predict the Output

```
def guess(n):
    print(n)
    return n

n = int(input("enter number: "))

print(guess(n-1))


n = guess(n) + guess(n+1)
print(n)
```

Output (n=4)

```
3
3
4
5
9
```

Functions Review: Predict the Output

```
def guess(n):
    print(n)
    return n+1

n = int(input("enter number: "))

print(guess(n-1))


n = guess(guess(n+1))
print(n)
```

Output (n=4)

```
3
4
5
6
7
```

Today's lecture

More on Recursions using:

1. **While loop**
2. **Functions**

Program to compute $\lfloor n^{1/2} \rfloor$

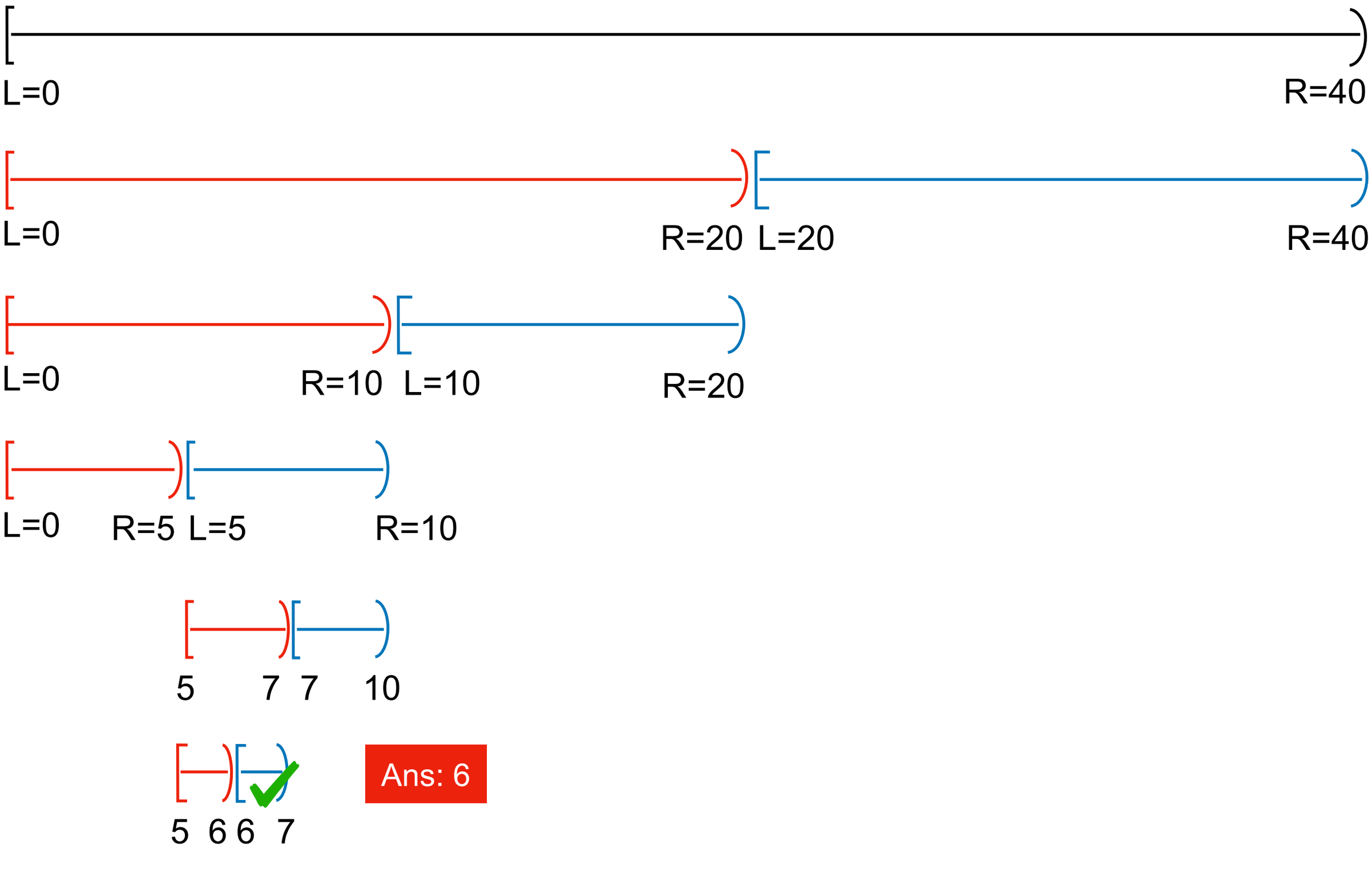
```
n = int(input())
i = 0
while ((i+1)**2 <= n):
   i = (i+1)
print(i)
```

Can we do better than scanning all values from 1,2,…,$\lfloor n^{1/2} \rfloor$?

Takes as input a positive integer n,

prints the largest y such that $y^2 \leqslant n$

Interval Halving to compute $\lfloor n^{1/2} \rfloor$ for n = 40



Computing $\lfloor n^{1/2} \rfloor$ using Interval Halving approach

```
n = int(input())

L = 0
R = n+1

while (L+1 != R):

    mid = (L+R)//2

    if (n < mid*mid):
        R = mid

    else:
        L = mid

print(L)
```

$\# \; L \leqslant n^{1/2} < R$

$\# \; L \leqslant n^{1/2} < mid$

$\# \; mid \leqslant n^{1/2} < R$

Can we compute $\lfloor n^{1/2} \rfloor$ using Recursions?

Program to compute $\lfloor n^{1/2} \rfloor$ using Functions

```
n = int(input())


def sqRoot(n, L, R):
    if L+1 == R:
        return ____          L
    mid = (L+R)//2
    if (n < mid*mid):
        return ____________sqRoot(n, L, mid)
    else:
        return ____________sqRoot(n, mid, R)


ans = sqRoot(n,0,n+1)
print(ans)
```

$\# \; L \leqslant n^{1/2} < R$

$\# \; L \leqslant n^{1/2} < mid$

$\# \; mid \leqslant n^{1/2} < R$

Program to compute $\lfloor n^{1/3} \rfloor$ using Functions

```
n = int(input())


def cubeRoot(n, L, R):
    if L+1 == R:
        return ____          L
    mid = (L+R)//2
    if (n < mid**3):
        return _____cubeRoot(n, L, mid)
    else:
        return _____cubeRoot(n, mid, R)


ans = cubeRoot(n,0,n+1)
print(ans)
```

$\# \; L \leqslant n^{1/3} < R$

$\# \; L \leqslant n^{1/3} < mid$

$\# \; mid \leqslant n^{1/3} < R$

Program to compute $\lfloor n^{1/2} \rfloor + \lfloor n^{1/5} \rfloor$

Trivial Approach: Define two functions sqRoot and fifthRoot.

```
def sqRoot(n, L, R):
    if L+1 == R:
        return ____          L
    mid = (L+R)//2
    if (n < mid**2):
        return _____________________sqRoot(n,L,mid)
    else:
        return _____________________sqRoot(n,mid,R)
```

```
def fifthRoot(n, L, R):
    if L+1 == R:
        return ____          L
    mid = (L+R)//2
    if (n < mid**5):
        return _____________________fifthRoot(n,L,mid)
    else:
        return _____________________fifthRoot(n,mid,R)
```

Can we avoid code repetition ?

Program to compute $\lfloor n^{1/2} \rfloor + \lfloor n^{1/5} \rfloor$

```
n = int(input())


def kthRoot(n, k, L, R):
    if L+1 == R:
        return _________          L
    mid = (L+R)//2
    if (n < mid**k):
        return _____________________kthRoot(n, k, L, mid)
    else:
        return _____________________kthRoot(n, k, mid, R)


ans = kthRoot(n,2,0,n+1) + kthRoot(n,5,0,n+1)
print(ans)
```

$\#\ L \leqslant n^{1/k} < R$

$\#\ L \leqslant n^{1/k} < mid$

$\#\ mid \leqslant n^{1/k} < R$